

Financial Earnings RAG

Crake AI Engineer Case Study — Lucca Pereira Martins

Architecture. CLI-first pipeline: PDFs under `data/reports/` are parsed with `pdfplumber` (page prose + each table as markdown), chunked (LangChain recursive split for prose; tables kept whole), embedded with OpenAI `text-embedding-3-small`, and stored in persistent Chroma (`data/index/`, cosine). `ask.py` runs a Strands agent (`gpt-5.4-nano`) with tools `retrieve_chunks`, `get_full_chunk`, and `think`, returning structured `AgentAnswer` (answer + citations, or abstain). The agent is a general financial research assistant over whatever is indexed. Company / doc type / a best-effort `doc_date` come from folder layout and filename regex at ingest—a stand-in for production CMS metadata. Optional FastAPI exposes `/ask` and `/search`; FastMCP mounts `ask`, `retrieve_chunks`, and `get_full_chunk` at `/mcp`.

Periods / fiscal calendars. Issuers do not share a fiscal calendar (e.g. Lam “Fiscal 2Q25” can be the calendar quarter ended Dec 2024; ASML “Q3 2024” is closer to Jul-Sep). We do *not* rewrite filenames or attach a normalised calendar-period field. Solution is retrieval + reading: chunk headers carry filename/doc_date, body text often states “quarter ended ...”, and the agent is told not to invent fiscal mappings. Ambiguous asks like “results 2025” (calendar vs fiscal) could be imperfect. An early design used an extra LLM call to extract the true reporting period from each PDF; we dropped that as expensive for something that should be structured metadata on the ingest path (ticker, report date, fiscal period labels) in production.

Model & cost. Agent: `gpt-5.4-nano`. Embeddings: `text-embedding-3-small`. List prices used: embeddings \$0.02 / 1M tokens; nano-class chat on the order of \$0.20 / \$1.25 per 1M input/output.

- **Bulk ingest** (~7.1k chunks, ~1.0M embedding tokens): ≈\$0.02.
- **Delta ingest** (one typical PDF, ~50–150 chunks): <<\$0.01 (embedding only).
- **Single ask** (query embed + tool rounds, incl. reasoning tokens): typically well under \$0.05.

Latency is driven by multi-step tool use, not embedding or retrieval.

Limitations. `pdfplumber` table/layout fragility; no OCR/vision; local Chroma; filename metadata is corpus-specific; **no shared fiscal↔calendar normalisation**; agent variance run-to-run.

Production / next. S3 + managed knowledge base (e.g. Bedrock KB) or managed vector DB; structured period metadata at ingest; OCR (e.g. Textract); separate ingest vs agent vs UI; optional graph / larger eval.

AI assistants. Cursor was used throughout (\$8.36 spent). Inline commentary and report writing by author + AI guidance.